Reinforcement learning from human feedback

SESSION 1 · FROM BASE MODELS TO RLHF

Pegah Alizadeh

pegahani.github.io

pegah.alizadeh@univ-lyon2.fr

Master's course, 2026–2027

Welcome

This course is about **how language models are trained to follow instructions and to match human preferences**, and about the statistics behind it.

By the end of the course, you will be able to:

- **Explain the pipeline:** instruction tuning, reward models, DPO and RL, with the theory behind each step
- **Model preferences:** fit and evaluate models of human comparisons, with honest uncertainty
- **Train models yourself:** small language models, on free GPUs
- **Do research:** design, run and report a small project of your own

The course at a glance

Session	When	Topic	In class
1	Week 1	From base models to RLHF	Lab: instruction tuning
2	Week 2	Preferences and reward models	Lab: a reward model for a language model
3	Week 2	RL for language models: PPO, GRPO	Lab: policy gradients for language models
4	Week 2	DPO and its noise-aware variants, project ideas	Lab: a small DPO run
5	Week 4	Over-optimization, AI feedback, open problems	Project feedback

Each session lasts 3 hours: a lecture with theory, a short break, then a lab of about an hour.

Where the materials come from

Two books, read together:

- **Lambert, *Reinforcement Learning from Human Feedback*** (rlhfbook.com): training language models in practice
- **Truong, Haupt, Koyejo, *Machine Learning from Human Preferences*** (mlhp.stanford.edu): the statistics of preferences

Also:

- **Slides:** adapted from Lambert's lectures (rlhfbook.com/course), with material from the second book and new derivations
- **Labs:** adapted from the code that accompanies Lambert's book
- **Homework:** written for this course, with its sources cited in each notebook
- **Figures from papers** keep their citations. Any errors are mine.

Reading plan

Session	Lambert, *RLHF*	Truong et al., *Machine Learning from Human Preferences*
1	Ch. 1, 3, 4	Introduction; Sections 1.1 and 1.2
2	Ch. 5, 10, 11	Ch. 1; Section 2.3
3	Ch. 8, 9	Ch. 2: preference learning for language models
4	Ch. 6, 7	Ch. 3: bandits
5	Ch. 12, 14, 15	Ch. 4 and 5: behavioral biases, aggregation

Further reading: T. Kaufmann et al., "A survey of reinforcement learning from human feedback", TMLR 2025.

How we work

- **Lecture, then lab:** each lab runs in your browser, on a free GPU. Work in pairs.
- **Homework at three levels:**
  - **Foundation:** everyone can complete it, with hints and solutions
  - **Core:** the level we aim for
  - **Deep:** derivations and open questions, if you want to go further
- **Low entry, earned depth:** you don't need any experience with language models. It's easy to get started, and the depth comes from doing the exercises.

The project

Your grade is based on the project only. Homework isn't graded, but it prepares you for the project.

Teams of 2 or 3, on a topic you choose, from a wide range of possibilities. We discuss ideas in session 4.

When	What
End of session 4	Discuss your idea with me
A few days later	One-page proposal: question, hypothesis, method, baseline, compute plan
Session 5	Early results and feedback
About 4 weeks after session 5	Report, code, and a 10-minute video
The week after	A 15-minute oral exam for each team

What counts: a clear question, a baseline, several runs with intervals, and honest reporting. A well-run negative result can earn full marks.

Class materials

What	Where
Main books	rlhfbook.com and mlhp.stanford.edu
Lambert's lectures and videos	rlhfbook.com/course
Book code	github.com/natolambert/rlhf-book
Labs and homework	github.com/pegahani/rlhf-course
Free GPUs	Google Colab or Kaggle: choose a T4 GPU
Gentle first reading	Illustrating RLHF, Hugging Face blog
If language models are new to you	Hugging Face LLM course

For today's lab: a Google or Kaggle account and a laptop.

Today

1. **Language models in practice:** the few tools we need
2. **The big picture:** what RLHF is and where it came from
3. **Instruction tuning:** chat templates and the completion mask
4. **Lab:** turn a base model into an assistant

You know RL and statistics already, so we'll spend our time on what's new: **language models, and what changes when the reward comes from people.**

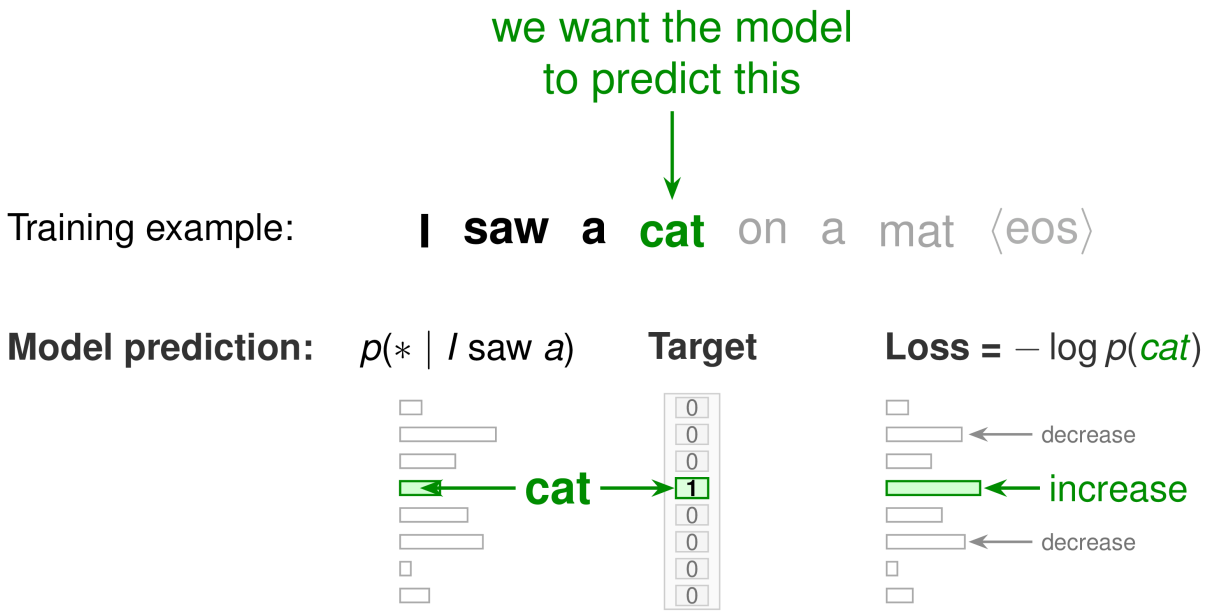
Language models in practice

What a language model is

A language model assigns **probabilities to text**, one **token** (a word or part of a word) at a time. By the chain rule,

$$P_\theta(x) = \prod_{t=1}^{T} P_\theta(x_t \mid x_1, \ldots, x_{t-1}).$$

This is called **autoregressive**. For this course, the model is a function from the tokens so far to next-token probabilities. Next, we open it once.



Next-token prediction: a decoder-only language model predicts one token at a time from the prefix before it.

Vaswani et al., 2017

Inside the model: one prediction, step by step

1. Tokens

The | cat | sat | on | the

Each token is an ID in a vocabulary of size V

→

2. Embed

Look up one learned vector per token: a row of a table

5 vectors, each with d numbers

→

3. Transformer layers × L

Each position mixes in information from **earlier** positions

Next slide

→

4. Hidden state h_t

The vector at the last position: a summary of "The cat sat on the"

$h_t \in \mathbb{R}^d$

→

5. Output head W

V rows × d columns

$z^{(t)} = W h_t$:

```
mat     9.1
floor   7.7
…
banana  0.4
```

e.g. 9.1 = $w_{\text{mat}}^\top h_t$

W is learned once and is the same for every text; h_t changes with the context, and so do the scores $z_j^{(t)} = w_j^\top h_t$ (illustrative values above):

- "The cat sat on the" gives an h_t close to w_{mat}, so **mat** scores high.
- "I drank a cup of" gives another h_t, close to w_{tea}, so **tea** scores high.

For example, OLMo-2 1B: $V \approx 100\text{K}$ tokens, vectors of length $d = 2{,}048$, $L = 16$ layers.

Vaswani et al., 2017

What one transformer layer does

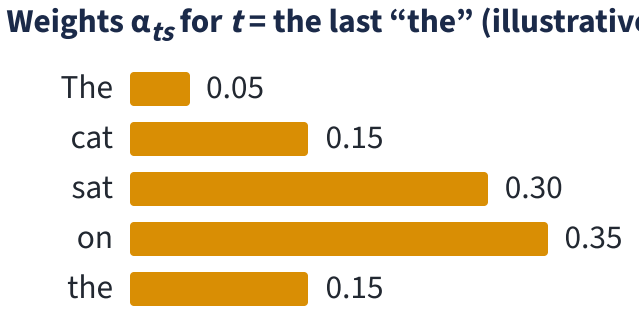
1. Attention: look back. Three learned matrices turn each vector h_s into three new vectors:

- **Query** $q_t = W_Q h_t$: what t looks for
- **Key** $k_s = W_K h_s$: what s offers
- **Value** $v_s = W_V h_s$: what s passes on

Position t adds the values, weighted by how well each key matches its query:

$$h_t \leftarrow h_t + \sum_{s \le t} \alpha_{ts}\, v_s$$

with $\alpha_{ts} = \mathrm{softmax}_{s \le t}(q_t^\top k_s)$.



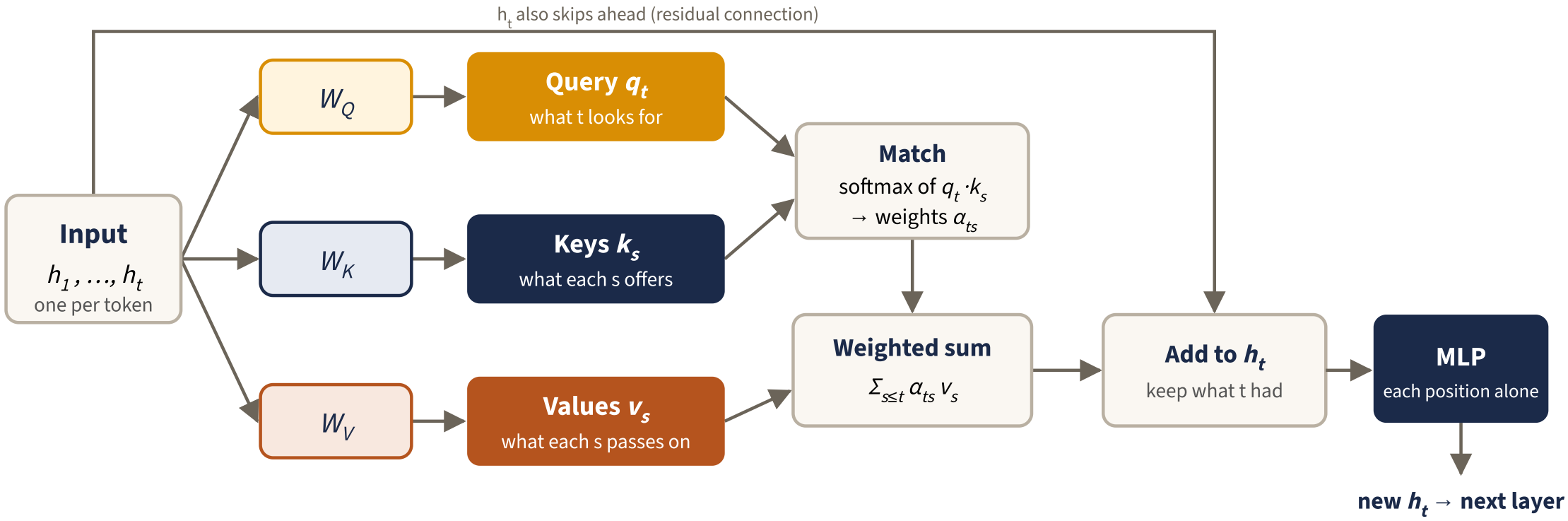
“sat on the” makes “mat” likely, so a useful layer looks there.

2. MLP: a small network then transforms each position on its own. **After L layers,** the last vector is the h_t that the output head scores.

Simplified: real layers use several attention heads, scale by $\sqrt{d}$, and normalize.

Vaswani et al., 2017

One layer as a picture: query, key, value



Like a library search: the **query** is your question, the **keys** are book labels, the **values** are the books' contents. You read each book in proportion to how well its label matches your question.

Vaswani et al., 2017

From scores to log-probabilities

Softmax turns the scores $z^{(t)}$, called **logits**, into a distribution:

$$P_\theta(x_t \mid x_{<t}) = \frac{e^{z^{(t)}_{x_t}}}{\sum_{j=1}^{V} e^{z^{(t)}_j}}.$$

The denominator sums over **all V tokens**, so probabilities add up to 1. **Only score differences matter:** mat is $9.1 - 7.7 = 1.4$ above floor, so $e^{1.4} \approx 4$ times as likely.

A sequence is a product of these terms, so in log-space it's a sum:

$$\log P_\theta(x) = \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}).$$

Logs are stable and easy to differentiate. **This sequence log-probability is used all course long.**

The loss at one position: cross-entropy

The **label** y_t is one-hot: 1 on the true next token, 0 elsewhere. Example: "The cat sat on the" → **mat**.

Token j	mat	floor	roof	…	banana
Label $y_t(j)$	**1**	0	0	…	0
Model $P_\theta(j \mid x_{<t})$	0.60	0.15	0.03	…	0.0001

Cross-entropy compares the two rows. Terms with label 0 vanish, so only the true token is left:

$$H(y_t, P_\theta) = -\sum_j y_t(j) \log P_\theta(j \mid x_{<t}) = -\log P_\theta(x_t \mid x_{<t}) = -\log 0.60 \approx 0.51$$

The more probability on the true token, the lower the loss: 0.01 on mat gives 4.6, and 1 gives 0.

Training is maximum likelihood

Over a whole text, sum the per-position losses, then average over the data. The sum is minus the sequence log-probability, so the loss is the **negative log-likelihood**:

$$\mathcal{L}_{\text{LM}}(\theta) = -\mathbb{E}_{x\sim\mathcal{D}}\left[\sum_{t=1}^{T}\log P_\theta(x_t \mid x_{<t})\right]$$

Minimizing the loss maximizes the likelihood of the data.

- **What it rewards:** the loss is lowest when the model predicts the true probabilities of text. Pretraining makes a model accurate, not helpful (Truong et al., 2025). If questions online are often followed by more questions, an accurate model does the same.
- **In SFT:** the same sum, but only over the response tokens. Next: how we pick them.

Three masks

Pretraining: two masks, on attention, kept in every later stage. Every real token counts in the loss.

- **Causal:** t sees only $s \leq t$. Built into the model.
- **Padding** (`attention_mask`): ignore the `<pad>` filler. Essential when padding comes first, as in batched generation. From the tokenizer.

Supervised fine-tuning adds a third, on the loss.

- **Completion** (`completion_mask`): only the response counts. **You build it yourself.**

If it's wrong, nothing crashes: you just train on the wrong tokens.

One training example, token by token:

Token	Padding (attention)	Completion (loss)
`<user>`	1	0
`2+2?`	1	0
`<assistant>`	1	0
`4`	1	**1**
`<end>`	1	**1**
`<pad>`	**0**	0

Why is `<end>` a 1? The model must learn to stop.

Putting it together: one training example as tensors

```
logits = model(input_ids, attention_mask=attention_mask).logits  # (B, T, V): scores
logits = logits[:, :-1, :]                # drop the last position: it has no next token to predict
labels = input_ids[:, 1:]                 # the target at each position is the actual next token
completion_mask = completion_mask[:, 1:]  # shift the mask in the same way (1 = response token)
log_probs = logits.log_softmax(dim=-1)    # softmax, in log-space, over the vocabulary
token_lp = log_probs.gather(-1, labels.unsqueeze(-1)).squeeze(-1)  # log-prob of each true next token
seq_lp = (token_lp * completion_mask).sum(dim=-1)                  # one log-prob per sequence: (B,)
loss = -seq_lp.mean()                                              # the SFT loss
```

B is the batch size, T the sequence length and V the vocabulary size. **The padding mask goes into the model; the completion mask goes into the sum.**

The **one-token shift** is the autoregressive step made literal: position t's logits predict token $t + 1$.

The big picture

A base model completes text

After pretraining we have a very capable autocomplete:

USER

The president of the United States in 2006 was

ASSISTANT (Llama 3.1 405B Base)

George W. Bush, the governor of Florida in 2006 was Jeb Bush, and John McCain was an Arizona senator in 2006…

Post-training makes it answer

The same prompt, after post-training:

USER

The president of the United States in 2006 was

ASSISTANT (Tülu 3 405B)

George W. Bush was the president of the United States in 2006. He served two terms in office, from January 20, 2001, to January 20, 2009.

The training stages, and where we study them

Stage	Learns from	What it changes	Session
Pretraining	Raw text; every token is graded	Knowledge and fluency	Background
Instruction tuning (SFT)	Demonstrations; only the response is graded	The question-and-answer format	**Today**
Preference tuning (RLHF)	Comparisons: response A is better than B	Whole responses, toward what people prefer	**2, 3, 4**
RL with verifiable rewards (RLVR)	A checker: is the answer right?	Answers that can be checked, such as math	**4**

Everything after pretraining is **post-training**: it moves the model's distribution over responses toward the behavior we want.

Open question: is post-training "just style transfer"? (Zhou et al., 2023)

Why RLHF?

In classical RL the reward is known: CartPole gives +1 per step. For many tasks it's **hard to write down**:

- It's easy to judge which poem is better, but hard to write a formula for "better"
- It's easy to spot a helpful answer, but hard to define "helpful"
- The most likely continuation of a text isn't always the most useful one

RLHF lets us optimize for behavior we can **evaluate**, even when we can't **specify** the reward.

Christiano et al., 2017; Ouyang et al., 2022; Ziegler et al., 2019

Preferences are everywhere

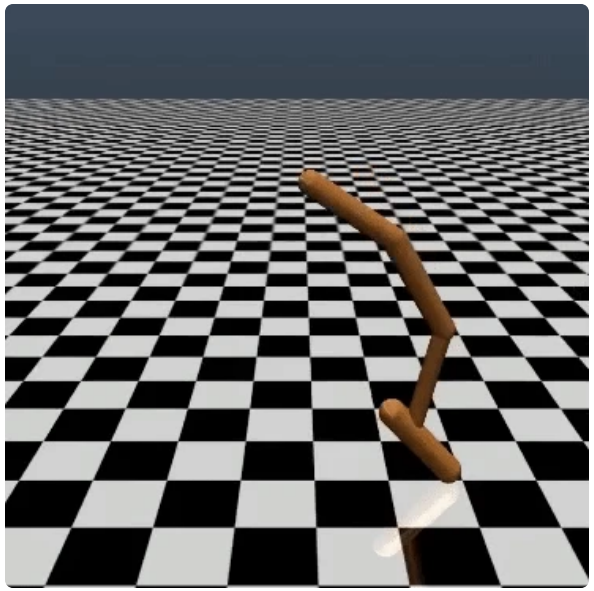
Language models are one example of a much older problem: **learning from comparisons**.

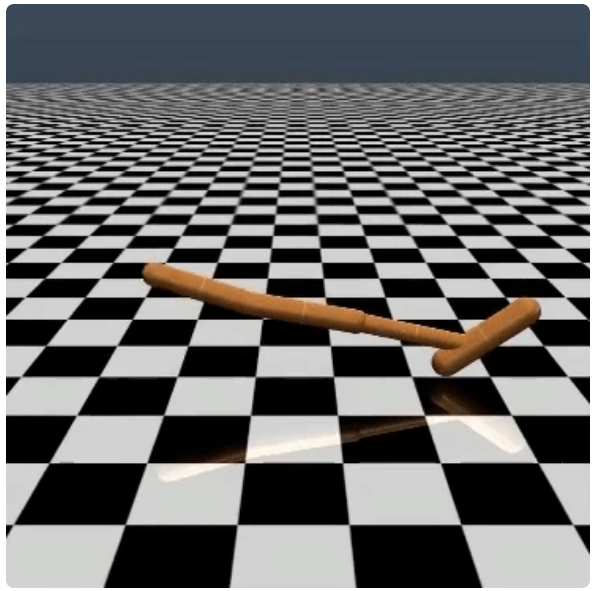
- **Recommender systems:** a click on item A rather than B suggests A $\succ$ B
- **Search:** clicking the third result suggests it beat the first two
- **Robotics:** people prefer smooth trajectories to jerky ones
- **Games and sports:** each match is a comparison: if A beats B, A is probably the stronger player
- **Language models:** annotators judge which response is more helpful, harmless or honest

These share **one mathematical structure**: comparisons or choices that reveal underlying preferences. In session 2 we'll see one simple model that covers all of them.

Truong et al., 2025

Which is the better backflip?





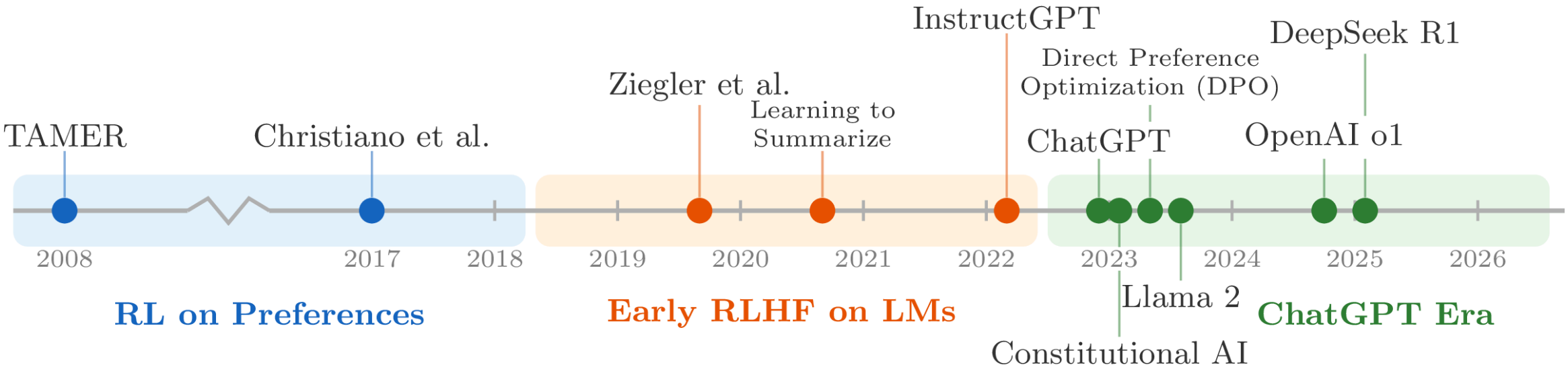
Left: human feedback. About 900 comparisons of short clips, then RL on the learned reward.

Right: a hand-written reward. A formula for "backflip", optimized directly.

Hard to write down, easy to judge: learn the reward from comparisons. That is RLHF.

Christiano et al., 2017

The path to modern RLHF



- **Ziegler** (Ziegler et al., 2019): RLHF on language models
- **InstructGPT** (Ouyang et al., 2022): canonical recipe
- **Constitutional AI** (Bai et al., 2022): AI feedback
- **DPO** (Rafailov et al., 2023): no separate reward model
- **Tülu 3** (Lambert et al., 2024): multi-stage recipe
- **DeepSeek R1** (Guo et al., 2025): popularized RLVR

What comparison data looks like

An annotator, human or AI, sees two responses to the same prompt and picks the better one.

USER

Explain why the sky is blue in one sentence.

ASSISTANT (Response A ✓)

The sky is blue due to Rayleigh scattering, where shorter blue wavelengths of sunlight are scattered more by atmospheric molecules than longer wavelengths.

USER

Explain why the sky is blue in one sentence.

ASSISTANT (Response B)

The sky appears blue because of the way light interacts with the atmosphere and stuff, it's basically just physics.

Types of comparison data

Preference data comes in several forms, for items $j \in \{1, \ldots, M\}$:

- **Full rankings:** $j_1 \succ j_2 \succ \cdots \succ j_M$. These are informative, but hard for people to give.
- **Choice from a set:** $(j, \mathcal{S})$, meaning j is preferred to everything else in $\mathcal{S}$
- **Binary comparisons:** $\mathcal{S} = \{j, j'\}$, written $Y_{jj'} = 1$. These are cheap and quick to collect, and RLHF uses them most.
- **Accept or reject:** a comparison with an outside option, "nothing"

Data can also depend on a **context**, such as a user, a goal or a prompt. In RLHF, one example is a triple (x, y_w, y_l): a prompt, a preferred response and a rejected one.

Truong et al., 2025

Why model preferences as random?

Data is noisy: the same person can answer differently on another day, and different people disagree. So we treat a preference as a **random order** over the items, with some distribution $p(\prec)$.

The usual assumptions are strong but practical:

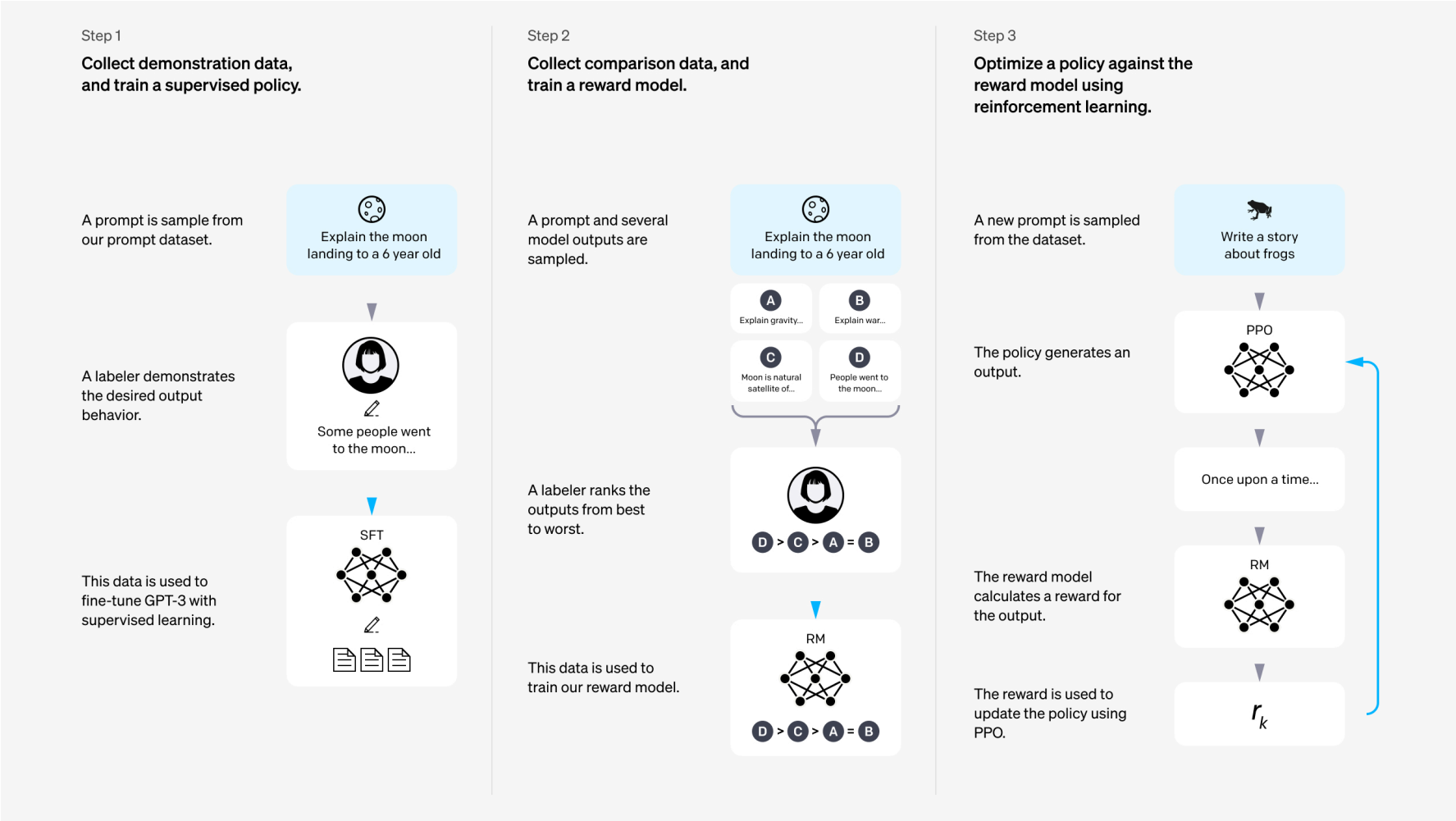
- **Totality:** any two items can be compared
- **Transitivity:** if $j \succ j'$ and $j' \succ k$, then $j \succ k$

A general distribution over orders of M items has $M! - 1$ parameters: 23 for $M = 4$, and over 3.6 million for $M = 10$. We need structure, and Bradley-Terry is the simplest.

People's preferences can also be **incomplete**, for example when helpfulness and safety pull in different directions. That's an absence of preference, not randomness.

Truong et al., 2025

InstructGPT's three-step recipe OPTIONAL



The three-step RLHF process from InstructGPT.

Ouyang et al., 2022

Step 1: instruction fine-tuning

The data: a list of N pairs $\mathcal{D} = \{(x_i, y_i^\star)\}_{i=1}^{N}$, a prompt and a response written by a person. InstructGPT used about 13K.

The loss: the log-likelihood from before, on the response tokens only:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{i=1}^{N}\sum_{t=1}^{|y_i^\star|} \log \pi_\theta\left(y_{i,t}^\star \mid x_i, y_{i,<t}^\star\right)$$

Two pairs from such a dataset (illustrative):

Prompt x	Response $y^\star$
Explain the moon landing to a 6-year-old.	Long ago, astronauts flew a big rocket to the Moon, walked on it, and came back home.
What is 2+2?	4

The response ends with `<end>`, so the model also learns to stop.

π_θ is a new name, not a new model: the same transformer, starting from the pretrained weights θ. In RL terms, the prompt is the state and the response the action.

Ouyang et al., 2022

Instruction tuning

What is instruction tuning?

Back to step 1 of the recipe. Everything that follows in the course, learning from preferences and RL, starts from an **instruction-tuned** model, and today's lab builds one.

Instruction tuning (SFT): keep training a base model on (prompt, ideal response) pairs.

	Pretraining	Instruction tuning
Data	raw text from the internet, trillions of tokens	conversations, thousands to millions of examples
Loss	next-token cross-entropy on every token	the same loss, on the **response tokens only**
Behavior	continues the text	**answers, then stops**

Example: to "What is the capital of France?", a base model may add "What is the capital of Germany? …"; a tuned one answers "Paris." and stops. It teaches the **format** of an answer, not which answer is better: preferences and RLHF add that.

Chat templates

A **chat template** turns a list of messages into the one flat token sequence the model sees:

```
<|im_start|>system
You are a friendly chatbot who always responds in the style of a pirate<|im_end|>
<|im_start|>user
How many helicopters can a human eat in one sitting?<|im_end|>
<|im_start|>assistant
```

- **Roles:** `system` holds instructions, `user` holds the person's messages, and `assistant` holds the model's responses
- **Generation cue:** the final `<|im_start|>assistant` tells the model that it's its turn
- **Stopping:** the model generates until it produces the end token, here `<|im_end|>`. A base model never learned this token; instruction tuning teaches it (next slide).

Each model family has its own format. Source: Lambert, Chapter 4.

What instruction tuning does differently

Same loss as pretraining. Three things change in the set-up:

① The completion mask (prompt masking): only the response counts in the loss, so the model learns to **answer**, not to write the user's side. Causal and padding masks: unchanged.

`<|im_start|>user` `What` `is` `2+2?` `<|im_end|>` `<|im_start|>assistant` `It's` `4.` `<|im_end|>`

■ masked: no loss ■ trained: learns to answer ■ `<|im_end|>` **is inside the trained region: learns to stop**

②③ Smaller batches, lower learning rate

OLMo 2	Pretraining	Instruction tuning
Batch	1024 × 4096 tokens	256 short prompts
Learning rate	3×10^{-4}	1×10^{-5} (**30× lower**)

- **Why? Small dataset:** 10K examples in batches of 1024 = ~10 updates. Smaller batches, more steps.
- **Low learning rate:** small steps, so the model **adapts without forgetting**.

OLMo et al., 2024

The completion mask on one example

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_t m_t \log \pi_{\boldsymbol{\theta}}(\text{token}_t \mid \text{previous}), \qquad m_t = 0 \text{ (prompt) or } 1 \text{ (response)}$$

$\boldsymbol{\theta}$: all the transformer's weights, **the only unknowns**. Each training step: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$.

Token to predict	P given previous	−log P	m	In the loss
`What`	0.05	3.00	0	0
`is`	0.30	1.20	0	0
`2+2?`	0.02	3.91	0	0
`<|im_end|>`	0.40	0.92	0	0
`<|im_start|>assistant`	0.90	0.11	0	0
`It's`	0.20	1.61	**1**	**1.61**
`4.`	0.70	0.36	**1**	**0.36**
`<|im_end|>`	0.10	2.30	**1**	**2.30**

Snapshot at the **current** θ (made-up values); each step changes them.

Loss $= 1.61 + 0.36 + 2.30 = \mathbf{4.27}$

Without the mask: ≈ 13.4, mostly the user's question.

- **No loss on the prompt:** its terms are $\times 0$. It is **read**, not **predicted**.
- **The end token costs most** ($P = 0.10$): raising it teaches the model **to stop**.

Why the model stops by itself

The end token is just one more token. At each step the model gives a probability to all V tokens, `<|im_end|>` included. Generation picks the most likely one, and the code has one rule: **stop when the picked token is the end token.**

Next token, after "The capital of France is Paris."	Base model	After SFT
`<|im_end|>`	0.02	**0.85 ← picked: stops**
new line	**0.30 ← picked: keeps writing**	0.05
"What"	0.15	0.01

Illustrative numbers.

What SFT changed: in every training example, `<|im_end|>` follows the complete answer and is **inside the trained region** (previous slide). Each step raises its probability there, until it becomes the **top choice** once the answer is complete.

What makes instruction tuning work OPTIONAL

Principles from Lambert, Chapter 4:

- **Data quality is key.** The completions are what the model actually learns from.
- **Small can be enough for chat.** Focused datasets can do well in narrow domains (Zhou et al., 2023). For example, No Robots has about 10K human-written samples.
- **Match the use case.** The best prompts resemble the tasks you care about downstream.
- **Later stages can recover from some noise.** The overall recipe matters more than any single stage.

Lab: from a base model to an assistant

Lab: what you'll do

Open the lab in Colab (or import the `.ipynb` from github.com/pegahani/rlhf-course into Kaggle) and choose a **T4 GPU**. Colab opens **your own copy**: use **File → Save a copy in Drive** to keep your work.

1. **Meet the base model.** Run a small pretrained model (1B parameters) on a few **prompts given in the notebook**. Does it answer, or just continue the text?
2. **Instruction-tune it.** Train on human-written conversations, with a chat template and a completion mask, for about 10 minutes on the free GPU.
3. **Compare before and after,** on the same prompts and on held-out prompts never seen in training: does it **answer**, and does it **stop**?

The given prompts (try your own too)

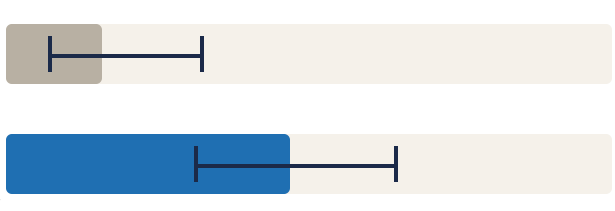
- What is the capital of France?
- Explain quantum computing in simple terms.
- Write a haiku about programming.
- How does photosynthesis work?

Work in pairs, especially if you get no GPU.

The exercises have hints and solutions.

What we saw

Measured: 32 held-out questions, each asked **separately**. For each answer: did the model **end it by itself** (end token), or was it cut off by the length limit?



The rest were cut off by the limit. Whiskers: 95% intervals.

Same questions, one by one: 13 stopped only after SFT, 3 only before. If SFT changed nothing, these 16 would split about 8–8; a 13–3 split has $p \approx 0.02$: **unlikely to be chance**.

How do your numbers compare?

Two of the given prompts, after SFT: **a good answer**, and one that **stops but ignores the task** (not a haiku).

USER
What is the capital of France?

ASSISTANT (After SFT)
The capital of France is Paris.

USER
Write a haiku about programming.

ASSISTANT (After SFT)
The best way to learn programming is to do it.

To discuss: did it learn to stop, or also something new? Is stopping quality?

Before next session

Homework 0, due before session 2: open it in Colab, or find `hw0-llm-basics.ipynb` at github.com/pegahani/rlhf-course. About 1.5 hours, no GPU needed.

What it practises: today's tools in small steps, starting from NumPy.

- softmax in PyTorch, and picking one log-probability per token
- the **log-probability of a response**, with a mask
- sampling and temperature, and chat templates

Why do it: the log-probability of a response, $\log \pi_\theta(y \mid x)$, is the quantity the rest of the course is built on. HW0 makes sure you can compute it in code before we build on it.

The notebook has checks and solutions: try first, then compare.

References (1/2)

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., et al. "*Constitutional ai: Harmlessness from ai feedback*." *arXiv preprint arXiv:2212.08073*, 2022. [link]

Christiano, P., Leike, J., Brown, T., Martic, M., Legg, S., et al. "*Deep Reinforcement Learning from Human Preferences*." *Advances in Neural Information Processing Systems*, 2017. [link]

Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., et al. "*Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning*." *arXiv preprint arXiv:2501.12948*, 2025. [link]

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., et al. "*Tülu 3: Pushing Frontiers in Open Language Model Post-Training*." *arXiv preprint arXiv:2411.15124*, 2024. [link]

OLMo, T., Walsh, P., Soldaini, L., Groeneveld, D., Lo, K., et al. "*2 OLMo 2 Furious*." *arXiv preprint arXiv:2501.00656*, 2024. [link]

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., et al. "*Training language models to follow instructions with human feedback*." *Advances in Neural Information Processing Systems*, 2022. [link]

Rafailov, R., Sharma, A., Mitchell, E., Manning, C., Ermon, S., et al. "*Direct preference optimization: Your language model is secretly a reward model*." *Advances in Neural Information Processing Systems*, 2023. [link]

Truong, S., Haupt, A., and Koyejo, S. "*Machine Learning from Human Preferences*." *Stanford University*, 2025. [link]

References (2/2)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., et al. "*Attention Is All You Need*." *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. [link]

Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., et al. "*LIMA: Less Is More for Alignment*." *Advances in Neural Information Processing Systems*, 2023. [link]

Ziegler, D., Stiennon, N., Wu, J., Brown, T., Radford, A., et al. "*Fine-tuning language models from human preferences*." *arXiv preprint arXiv:1909.08593*, 2019. [link]